Chapter 4

BRUTE FORCE ALGORITHM

Brute force approach

- Brute force is a straightforward approach to solving a problem, usually directly based on the problem statement and definitions of the concepts involved.
- A brute force approach is an approach that finds all the possible solutions to find a satisfactory solution to a given problem. The brute force algorithm tries out all the possibilities till a satisfactory solution is not found.
- The time complexity of a brute force algorithm is often proportional to the input size.
- Unlike other design techniques, brute-force can be applied to a very wide variety of problems

- if only a few instances or small-size instances of a problem need to be solved, brute-force can lift the burden of designing more efficient algorithms
- Brute-force can serve as a reference point when judging the efficiency of other alternatives
- Brute force algorithms are simple and consistent, but very slow.

Such an algorithm can be of two types:

- **Optimizing:** In this case, the best solution is found. To find the best solution, it may either find all the possible solutions to find the best solution or if the value of the best solution is known, it stops finding when the best solution is found.

  For example: Finding the best path for the travelling salesman problem. Here best path means that travelling all the cities and the cost of travelling should be minimum.

- **Satisficing:** It stops finding the solution as soon as the satisfactory solution is found.

  For example, finding the travelling salesman path which is within 10% of optimal.

- Often Brute force algorithms require exponential time. Various heuristics and optimization can be used:
- **Heuristic:** A rule of thumb that helps you to decide which possibilities we should look at first.
- **Optimization:** A certain possibilities are eliminated without exploring all of them.

Advantages of a brute-force algorithm

The following are the advantages of the brute-force algorithm:

- This algorithm finds all the possible solutions, and it also guarantees that it finds the correct solution to a problem.
- This type of algorithm is applicable to a wide range of domains.
- It is mainly used for solving simpler and small problems.
- It can be considered a comparison benchmark to solve a simple problem and does not require any particular domain knowledge.

Disadvantages of a brute-force algorithm

The following are the disadvantages of the brute-force algorithm:

- It is an inefficient algorithm as it requires solving each and every state.
- It is a very slow algorithm to find the correct solution as it solves each state without considering whether the solution is feasible or not.
- The brute force algorithm is neither constructive nor creative as compared to other algorithms.

Sequential Search

- brute-force approach is applicable for searching and sorting porblem. Here we discuss two applications of this strategy to the problem of searching.
- We have already encountered a brute-force algorithm for the general searching problem: it is called sequential search.
- To repeat, the algorithm simply compares successive elements of a given list with a given search key until either a match is encountered (successful search) or the list is exhausted without finding a match (unsuccessful search).
- A simple extra trick is often employed in implementing sequential search: if we append the search key to the end of the list, the search for the key will have to be successful, and therefore we can eliminate the end of list check altogether.

Here is pseudocode of this enhanced

```
SequentialSearch2(A[0..n], K)
//Implements sequential search with a search key as a sentinel
//Input: An array A of n elements and a search key K and Output: The index of the first element in A[0..n − 1] whose value is
// equal to K or −1 if no such element is found
A[n]← K
i ← 0
while A[i] ≠ K do
i ← i + 1
if i<n return i
else return −1
```

- Another straightforward improvement can be incorporated in sequential search if a given list is known to be sorted: searching in such a list can be stopped as soon as an element greater than or equal to the search key is encountered.
- Sequential search provides an excellent illustration of the brute-force approach, with its characteristic strength (simplicity) and weakness (inferior efficiency).
- The efficiency results obtained for the standard version of sequential search change for the enhanced version only very slightly, so that the algorithm remains linear in both the worst and average cases.

Brute-Force String Matching

- given a string of **n** characters called the **text** and a string of **m** characters ($m \leq n$) called the **pattern**, find a substring of the text that matches the pattern.
- To put it more precisely, we want to find **i** the **index** of the leftmost character of the first matching substring in the **text** such that $t_i = p_0, \ldots, t_{i+j} = p_j, \ldots, t_{i+m-1} = p_{m-1}$

$$
\begin{array}{ccccccccccc}
t_0 & \cdots & t_i & \cdots & t_{i+j} & \cdots & t_{i+m-1} & & \cdots & t_{n-1} & \text{text } T \\
 & & \updownarrow & & \updownarrow & & \updownarrow & & & & \\
 & & p_0 & \cdots & p_j & \cdots & p_{m-1} & & \text{pattern } P & &
\end{array}
$$

- If matches other than the first one need to be found, a string-matching algorithm can simply continue working until the entire text is exhausted.

- A brute-force algorithm for the string-matching problem is quite obvious:
- align the pattern against the first **m** characters of the text and start matching the corresponding pairs of characters from left to right until either all the **m** pairs of the characters match (then the algorithm can stop) or a mismatching pair is encountered.
- In the latter case, shift the pattern one position to the right and resume the character comparisons, starting again with the first character of the pattern and its counterpart in the text.
- Note that the last position in the text that can still be a beginning of a matching substring is **n − m**(provided the text positions are indexed from 0 to n − 1).
- Beyond that position, there are not enough characters to match the entire pattern; hence, the algorithm need not make any comparisons there.

```
ALGORITHM BruteForceStringMatch(T [0..n − 1], P[0..m − 1])
//Implements brute-force string matching
//Input: An array T [0..n-1] of n characters representing a text and
// an array P[0..m -1] of m characters representing a pattern
//Output: The index of the first character in the text that starts a
// matching substring or −1 if the search is unsuccessful
j ← 0
while j<m and P[j ] = T [i + j ] do
j ← j + 1
if j = m return i
return −1
```

- The worst case is much worse: the algorithm may have to make all **m** comparisons before shifting the pattern, and this can happen for each of the **n − m + 1** tries.

```
N O B O D Y _ N O T I C E D _ H I M
N O T
  N O T
    N O T
      N O T
        N O T
          N O T
            N O T
              N O T
```

- **$m(n - m + 1)$** character comparisons, which puts it in the **$O(nm)$** class.
- For a typical word search in a natural language text, however, we should expect that most shifts would happen after very few comparisons (check the example again).
- Therefore, the average-case efficiency should be considerably better than the worst-case efficiency.
- There are several more sophisticated and more efficient algorithms for string searching.

Closest-pair problem

- The closest-pair problem calls for finding the two closest points in a set of **n** points. It is the simplest of a variety of problems in computational geometry that deals with proximity of points in the plane or higher-dimensional spaces.
- Points in question can represent such physical objects as airplanes or post offices as well as database records, statistical samples, DNA sequences, and so on.
- An air-traffic controller might be interested in two closest planes as the most probable collision candidates.
- A regional postal service manager might need a solution to the closest pair problem to find candidate post-office locations to be closed.

- One of the important applications of the closest-pair problem is cluster analysis in statistics. Based on **n** data points, hierarchical cluster analysis seeks to organize them in a hierarchy of clusters based on some similarity metric.
- For numerical data, this metric is usually the **Euclidean distance**; for text and other nonnumerical data, metrics such as the **Hamming distance** are used.
- A bottom-up algorithm begins with each element as a separate cluster and merges them into successively larger clusters by combining the closest pair of clusters.
- For simplicity, we consider the two-dimensional case of the closest-pair problem.
- We assume that the points in question are specified in a standard fashion by their (x, y) Cartesian coordinates and that the distance between two points pi(xi, yi) and pj (xj , yj) is the standard Euclidean distance

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

The brute-force approach to solving this problem leads to the following obvious algorithm: compute the distance between each pair of distinct points and find a pair with the smallest distance. Of course, we do not want to compute the distance between the same pair of points twice. To avoid doing so, we consider only the pairs of points (pi, pj) for which i<j .

- Pseudocode below computes the distance between the two closest points; getting the closest points themselves requires just a trivial modification.

ALGORITHM BruteForceClosestPair(P)

//Finds distance between two closest points in the plane by brute force

//Input: A list P of n ($n \geq 2$) points p1(x1, y1), . . . , pn(xn, yn)

//Output: The distance between the closest pair of points

$d \leftarrow \infty$

for $i \leftarrow 1$ to $n - 1$ do

for $j \leftarrow i + 1$ to n do

$d \leftarrow \min(d, \text{sqrt}((xi - xj)^2 + (yi - yj)^2))$ //sqrt is square root

return d

- computing square roots in the loop can be avoided! (Can you think how?) The trick is to realize that we can simply ignore the square-root function and compare the values $(x_i - x_j)^2 + (y_i - y_j)^2$ themselves. We can do this because the smaller a number of which we take the square root, the smaller its square root, or, as mathematicians say, the square-root function is strictly increasing.
- Then the basic operation of the algorithm will be squaring a number. The number of times it will be executed can be computed as follows:

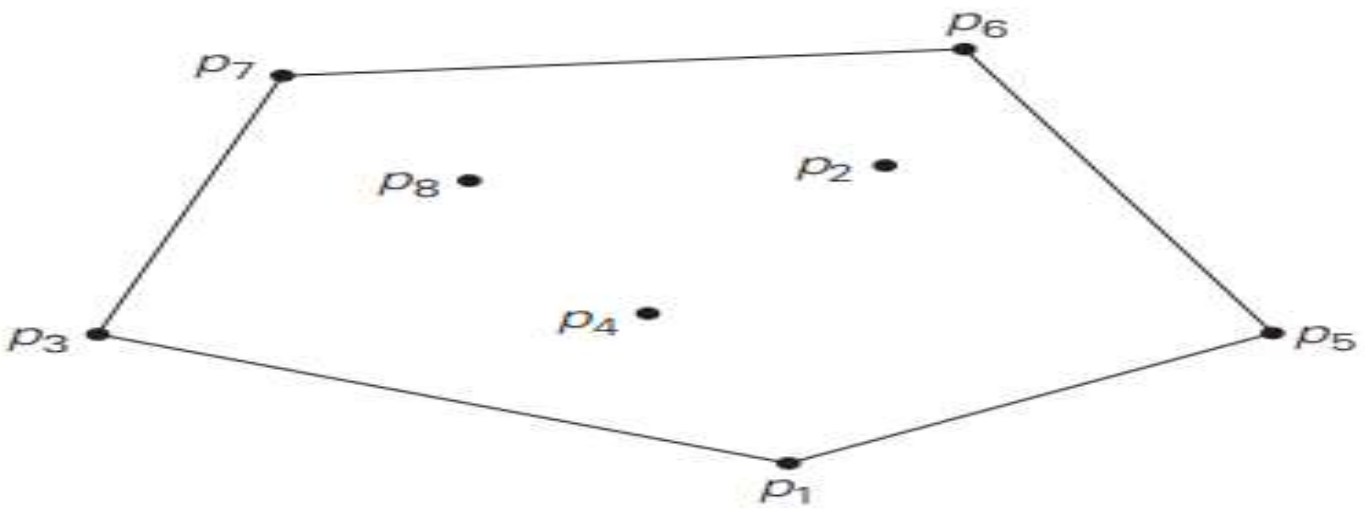
$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 2 = 2 \sum_{i=1}^{n-1} (n - i)$$

$$= 2[(n-1) + (n-2) + \cdots + 1] = (n-1)n \in \Theta(n^2).$$

Convex-Hull Problem

- The convex hull of the set of points Q is the convex polygon P that encompasses all of the points given. The problem of finding the smallest polygon P such that all the points of set Q are either on the boundary of P or inside P is known as the convex hull problem.
- Finding the convex hull for a given set of points in the plane or a higher dimensional space is one of the most important—some people believe the most important—problems in computational geometry.This prominence is due to a variety of applications in which this problem needs to be solved, either by itself or as a part of a larger task.
- Several such applications are based on the fact that convex hulls provide convenient approximations of object shapes and data sets given.

For example, in computer animation, replacing objects by their convex hulls speeds up collision detection; the same idea is used in path planning for Mars mission rovers.



The convex hull for this set of eight points is the convex polygon with vertices at p1, p5, p6, p7, and p3

- Convex hulls are used in computing accessibility maps produced from satellite images by Geographic Information Systems.
- They are also used for detecting outliers by some statistical techniques. An efficient algorithm for computing a diameter of a set of points,which is the largest distance between two of the points, needs the set’s convex hull to find the largest distance between two of its extreme points.
- Finally,convex hulls are important for solving many optimization problems, because their extreme points provide a limited set of solution candidates.

- Brute force approach for the convex hull problem involves considering all possible combinations of points and checking if they form a convex polygon.
- The basic idea is to iterate through all possible pairs of points and check if all other points lie within the polygon formed by those two points. If they do, add those two points to the convex hull.

The steps for the brute force approach are as follows:

1. Iterate through all possible pairs of points (p1, p2) in the given set.
2. For each pair (p1, p2), iterate through all other points in the set.
3. Check if each point p is on the same side of the line formed by (p1, p2) using the cross product. If any point lies on the opposite side, it is not part of the convex hull.
4. 4. If all points lie on the same side, add p1 and p2 to the convex hull.

5. Repeat steps 1-4 for all pairs of points.

6. The final convex hull will consist of all the points that have been added during the iterations.

- The brute force approach for the convex hull problem has a time complexity of O(n^4), where n is the number of points in the input set. This is because we need to iterate through all possible pairs of points (O(n^2)) and for each pair, check the side of each remaining point(O(n)) using the cross product (O(1) operation).
- Although this approach is relatively simple to understand and implement, it is highly inefficient for large input sizes due to its exponential time complexity. Hence, more specialized algorithms like gift wrapping, Graham's scan, or Quickhull are preferred for practical applications of the convex hull problem.

Traveling Salesman Problem

- The traveling salesman problem (TSP) has been intriguing researchers for the last 150 years by its seemingly simple formulation, important applications, and interesting connections to other combinatorial problems.
- In layman's terms, the problem asks to find the shortest tour through a given set of **n** cities that visits each city exactly once before returning to the city where it started.
- The problem can be conveniently modeled by a weighted graph, with the graph's vertices representing the cities and the edge weights specifying the distances. Then the problem can be stated as the problem of finding the shortest Hamiltonian circuit of the graph.
- (A Hamiltonian circuit is defined as a cycle that passes through all the vertices of the graph exactly once.

- The brute force traveling salesman algorithm is a method used to find the shortest possible path that a salesman can take in a given set of cities. It is a very simple algorithm that involves generating all possible permutations of cities and calculating the distance between each pair of cities.
- The algorithm starts by selecting a starting city, say city A. The algorithm then generates all possible permutations of the remaining cities.

  For example, if there are five cities, the algorithm generates all possible orders in which the cities could be visited, which is 5! = 120 permutations. Then, for each permutation, the algorithm calculates the total distance traveled by the salesman.
- Once all the permutations have been generated and the total distance traveled has been calculated for each permutation, the algorithm selects the permutation that results in the shortest distance traveled.

- The brute force traveling salesman algorithm is a very time-consuming algorithm and is only practical for small numbers of cities. As the number of cities increases, the number of possible permutations increases exponentially, making the algorithm impractical.

  For example, if there are 15 cities to be visited, there are 15! = 1.3 x 10^12 possible permutations, which would take an impractical amount of time to calculate.

- **Brute force Knapsack Problem** : reading assigment

Thanks